Mittuniversitetet
MID SWEDEN UNIVERSITY

DT081A – Signal and Image Processing

Lecture 9 - Spatial domain image processing II



Roger Olsson (PhD)
Department of Computer and Electrical Engineering

Image processing

A digital image can be represented as a matrix like the following

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(N-1,0) & f(N-1,1) & \cdots & f(N-1,N-1) \end{bmatrix}$$

with

$$0 \leq f(x,y) \leq G-1$$

where usually N and G are expressed as integer powers of 2

$$N = 2^n, \quad G = 2^m$$

Image processing, image transformations and operators

- Image Processing is performed using Image Transformations
- Image Transformations are performed using Operators
- Operator takes an image as input and produces another image

Operators

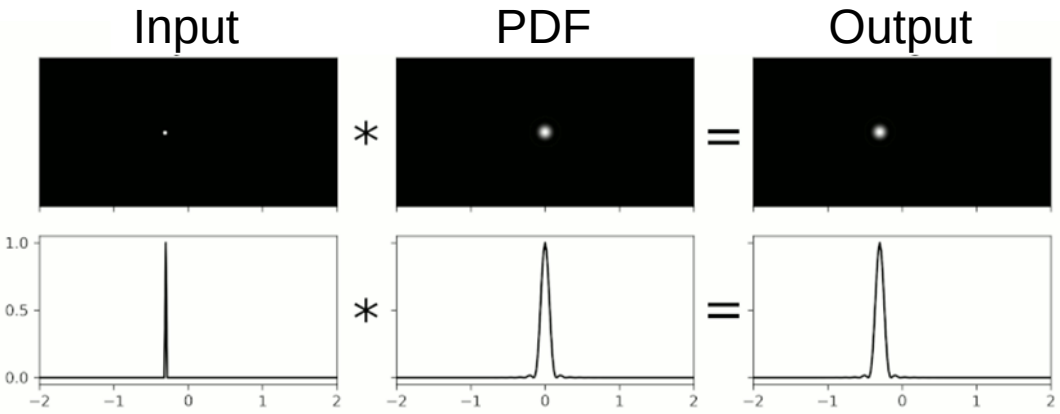
An operator is defined using its point spread function (PSF)

$$O[\text{point source}] = \text{point spread function}$$

or

$$O[\delta(x-\alpha, y-\beta)] = h(x, y, \alpha, \beta)$$

where $\delta(x-\alpha, y-\beta)$ is a point source of brightness 1 centered at point (α, β)



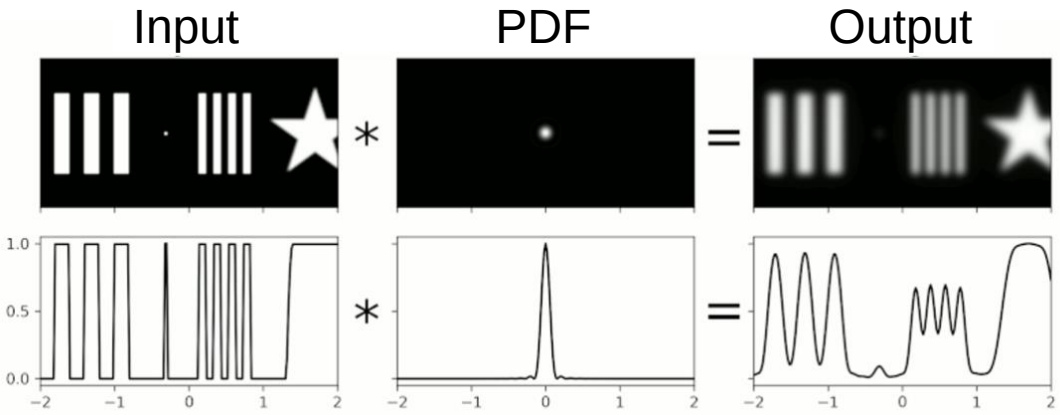
Other words for the PSF are impulse response, kernel, blur function, and more.

Point spread function (PDF)

For a linear operator, when the point side is a times brighter, the result will be a times larger

$$O[a\delta(x-\alpha, y-\beta)] = ah(x, \alpha, y, \beta)$$

An image is a collection of scaled point sources, pixels with individual brighness.



O is linear if

$$O[af + bg] = aO[f] + bO[g]$$

For all images f and g and all scalars a and b

An image is the sum of these point sources

Operator effect

The effect of an operator defined using a PSF $h(x, y, \alpha, \beta)$ on and image $f(x, y)$ can be written as

$$g(\alpha, \beta) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(x, y, \alpha, \beta)$$

which produces the output image $g(\alpha, \beta)$

The PSF expresses how much the input value at position (x, y) influences the output at position (α, β)

Shift-invariant PSF

If the influence of the PSF is independent of absolute position, and only on the relative position of influencing and influenced pixels, the PSF is shift-invariant

$$h(x, \alpha, y, \beta) = h(\alpha - x, \beta - y)$$

Then

$$g(\alpha, \beta) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(x, y, \alpha, \beta) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(\alpha - x, \beta - y)$$

which is a convolution!

Separable

If the image's columns are influenced independently from the rows, the PSF is separable

$$h(x, \alpha, y, \beta) = h_c(x, \alpha) h_r(y, \beta)$$

The output image can then be written as

$$g(\alpha, \beta) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(x, y, \alpha, \beta) = \sum_{x=0}^{N-1} h_c(x, \alpha) \sum_{y=0}^{N-1} f(x, y) h_r(y, \beta)$$

Matrix formulation

Lets rewrite previous equation with the help of matrices

$$g(\alpha,\beta) = f(0,0)h(0,\alpha,0,\beta) + f(1,0)h(1,\alpha,0,\beta) + \cdots + f(N-1,0)h(N-1,\alpha,0,\beta) + \cdots$$
$$+ f(0,1)h(0,\alpha,1,\beta) + \cdots + f(N-1,N-1)h(N-1,\alpha,N-1,\beta)$$

The right hand side can be thought of as the dot product of vector $\mathbf{h}^T_{\alpha,\beta}$ and vector $\mathbf{f}^T$

$$\mathbf{h}^T_{\alpha\beta} \equiv [h(0,\alpha,0,\beta), h(1,\alpha,0,\beta), \ldots, h(N-1,\alpha,0,\beta), \ldots, h(N-1,\alpha,N-1,\beta)]$$
$$\mathbf{f}^T \equiv [f(0,0), f(1,0), \ldots, f(N-1,0), \ldots, f(0,1), \ldots, f(N-1,N-1)]$$

$\mathbf{f}^T$ is the image written as a vector by stacking each column under the other

Stacking (motivating $f = \mathbf{f}^{\mathrm{T}}$)

The stacking operation allows for transformation between an NxN image f and an N^2x1 vector $\mathbf{f}^{\mathrm{T}}$

Lets define some vectors $\mathbf{V}_n$ and matrices N_n as

$$\mathbf{V}_n = \begin{pmatrix} 0 \\ \vdots \\ 1\ (\text{row } n) \\ \vdots \\ 0 \end{pmatrix} \qquad N_n = \begin{pmatrix} \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \cdots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \\ \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \cdots & \mathbf{0} \end{pmatrix}$$

where the dimensions are $Nx1$ and N^2xN respectively

With the defined variables and matrices we can express the vector $\mathbf{f}$, which corresponds to the square matrix f, as

$$\mathbf{f} = \sum_{n=1}^{N} N_n f \mathbf{V}_n$$

and vice versa

$$f = \sum_{n=1}^{N} N_n^T \mathbf{f} \mathbf{V}_n^T$$

Stacking example

A $3x3$ image f is defined as

$$f = \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix}$$

Use stacking to write f in vector form $\mathbf{f}$

- Define vectors $\mathbf{V}_1, \mathbf{V}_2, \mathbf{V}_3$ and matrices N_1, N_2, N_3 as

$$\mathbf{V}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{V}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{V}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$N_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad N_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad N_3 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Stacking example cont.

$$\mathbf{f} = \sum_{n=1}^{N} N_n f \mathbf{V}_n = N_1 \mathbf{f} \mathbf{V}_1 + N_2 \mathbf{f} \mathbf{V}_2 + N_3 \mathbf{f} \mathbf{V}_3$$

Calculate each term separately

$$N_1 \mathbf{f} \mathbf{V}_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} f_{11} \\ f_{21} \\ f_{31} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Similarly

$$N_2 f \mathbf{V}_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ f_{12} \\ f_{22} \\ f_{32} \\ 0 \\ 0 \\ 0 \end{pmatrix} \qquad N_3 f \mathbf{V}_3 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ f_{13} \\ f_{23} \\ f_{33} \end{pmatrix}$$

Summing the terms gives the vector

$$\mathbf{f} = \begin{pmatrix} f_{11} \\ f_{21} \\ f_{31} \\ f_{12} \\ f_{22} \\ f_{32} \\ f_{13} \\ f_{23} \\ f_{33} \end{pmatrix}$$

Matrix formulation returned

$$\mathbf{h}_{\alpha\beta}^{T} \equiv [h(0,\alpha,0,\beta), h(1,\alpha,0,\beta), \ldots, h(N-1,\alpha,0,\beta), \ldots, h(N-1,\alpha,N-1,\beta)]$$

$$\mathbf{f}^{T} \equiv [f(0,0), f(1,0), \ldots, f(N-1,0), \ldots, f(0,1), \ldots, f(N-1,N-1)]$$

Fundamental equation of linear image processing

Writing g in the same way as f (stacked) will arrange vectors $h^T_{\alpha,\beta}$ as rows in a matrix H and

$$\mathbf{g} = H\mathbf{f}$$

where H is a square $N^2 x N^2$ matrix consisting of NxN submatrices arranged as

$$H = \begin{pmatrix} \alpha\begin{pmatrix} y=0 \\ \beta=0 \end{pmatrix} & \alpha\begin{pmatrix} y=1 \\ \beta=0 \end{pmatrix} & \cdots & \alpha\begin{pmatrix} y=N-1 \\ \beta=0 \end{pmatrix} \\ \alpha\begin{pmatrix} y=0 \\ \beta=1 \end{pmatrix} & \alpha\begin{pmatrix} y=1 \\ \beta=1 \end{pmatrix} & \cdots & \alpha\begin{pmatrix} y=N-1 \\ \beta=1 \end{pmatrix} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha\begin{pmatrix} y=0 \\ \beta=N-1 \end{pmatrix} & \alpha\begin{pmatrix} y=1 \\ \beta=N-1 \end{pmatrix} & \cdots & \alpha\begin{pmatrix} y=N-1 \\ \beta=N-1 \end{pmatrix} \end{pmatrix}$$

Separable transform in matrix form

$$g(\alpha,\beta) = \sum_{x=0}^{N-1} h_c(x,\alpha) \sum_{y=0}^{N-1} f(x,y) h_r(y,\beta)$$

Notice that the last factor represents the product of two NxN matrices, resulting in an NxN matrix. Lets define that as

$$s(x,\beta) = \sum_{y=0}^{N-1} f(x,y) h_r(y,\beta) = fh_r$$

which allows the output matrix to be written

$$g(\alpha,\beta) = \sum_{x=0}^{N-1} h_c(x,\alpha) s(x,\beta) = h_c^T s$$

that in matrix form becomes

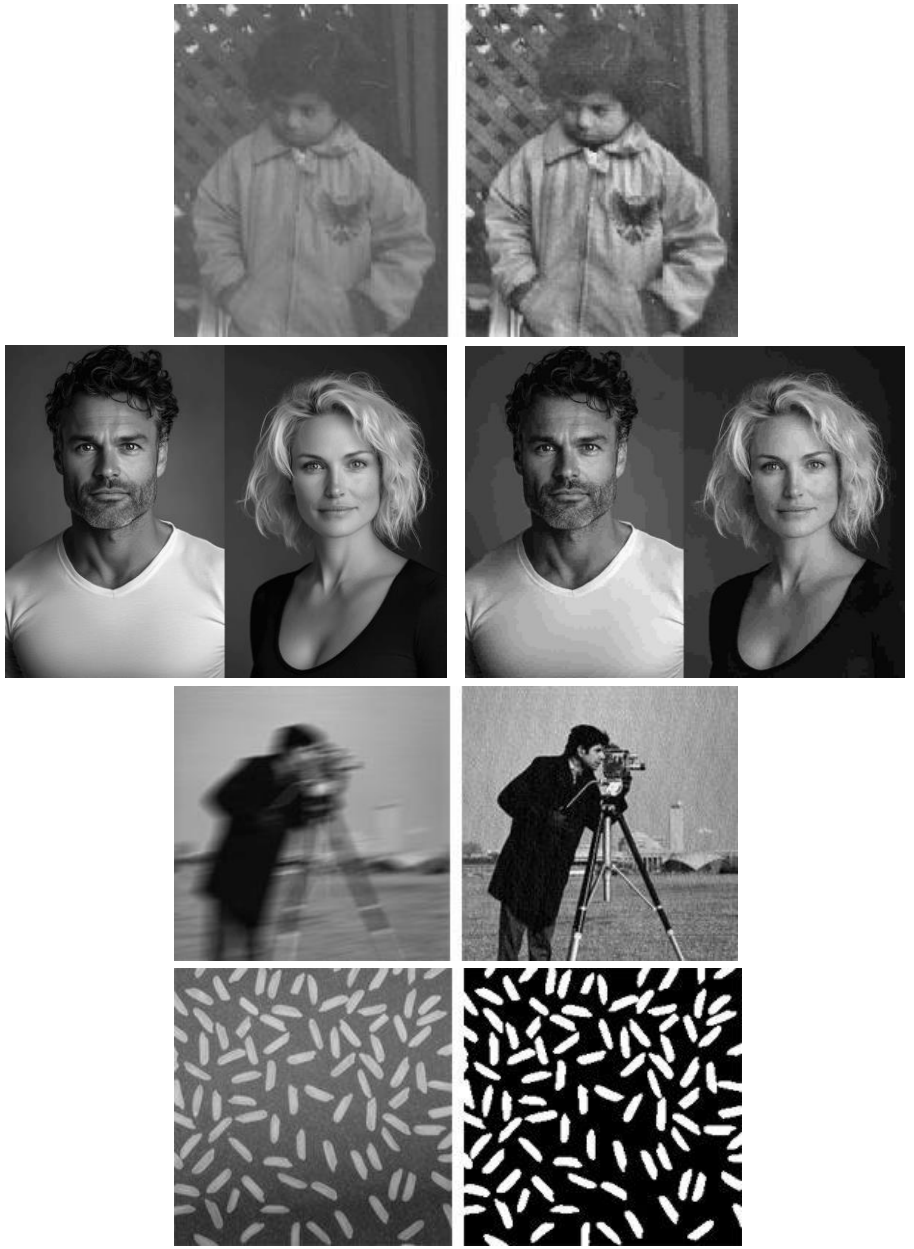
$$\mathbf{g} = h_c^T f h_r$$

- What does this mean?

Separability implies that operator O operates on the rows of image f independently from its columns using two different matrices h_r and h_c respectively

Different image processing categories

- **Image enhancement**
  Given an image f, choose h_r and h_c, such that output image g is better than f
- **Image compression**
  Given an image f, choose h_r and h_c, such that output image g can be represented with fewer bits than f without loosing quality
- **Image restoration**
  Given an image g and an estimate of $h(x, y, \alpha, \beta)$, determine image f
- **Image pre-processing for computer vision**
  Given an image f, choose h_r and h_c, so that the output image g enhances certain features of within f



Pixel based processing

$$g(i,j) = h\big(f(i,j)\big)$$

Change contrast and brightness change:
$g = c \cdot f + b$



g(i,j) = 1*f(i,j) + 0



g(i,j) = 1.5*f(i,j) + 0



g(i,j) = 2*f(i,j) + 0



g(i,j) = 1*f(i,j) + 0.25



g(i,j) = 1.5*f(i,j) + 0.25



g(i,j) = 2*f(i,j) + 0.25



g(i,j) = 1*f(i,j) + 0.5



g(i,j) = 1.5*f(i,j) + 0.5



g(i,j) = 2*f(i,j) + 0.5

Linear filter examples



original



blurred

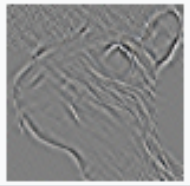
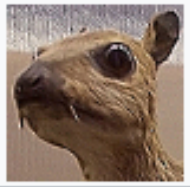


sharpening



Edge-preserving smoothing

Linear filter examples with kernels

Operation	Kernel ω	Image result g(x,y)
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Operation	Kernel ω	Image result g(x,y)
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 × 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

DEMO

Convolution in practice

- If we have two matrices, the first a kernel h, and the second an image f, convolution is the process of flipping both the rows and columns of the kernel and multiplying locally similar entries and summing

$$g(\alpha, \beta) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(\alpha - x, \beta - y)$$

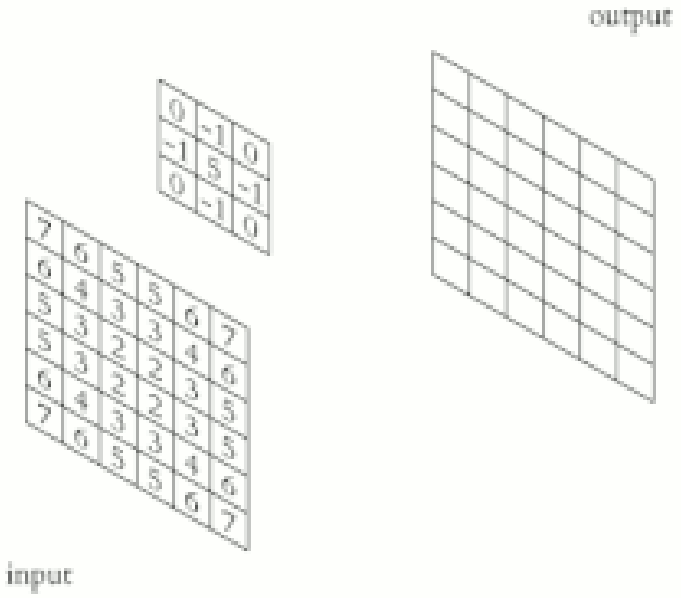
```
for each image row in input image:
    for each pixel in image row:

        set accumulator to zero

        for each kernel row in kernel:
            for each element in kernel row:

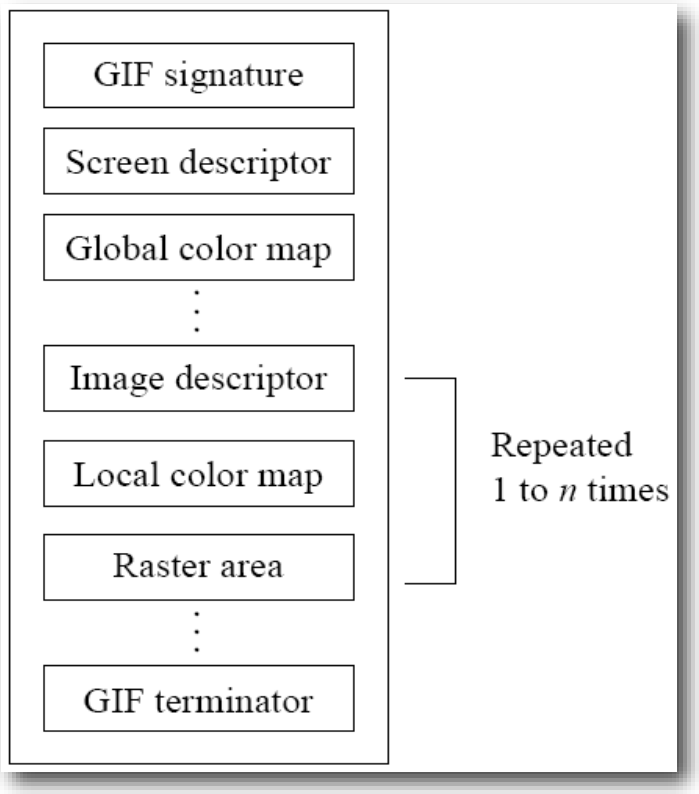
                if element position  corresponding* to pixel position then
                    multiply element value  corresponding* to pixel value
                    add result to accumulator
                endif

         set output image pixel to accumulator
```



GIF - Graphics Interchange Formatrmat

- For the standard specification, the general file format of a GIF87 file is as

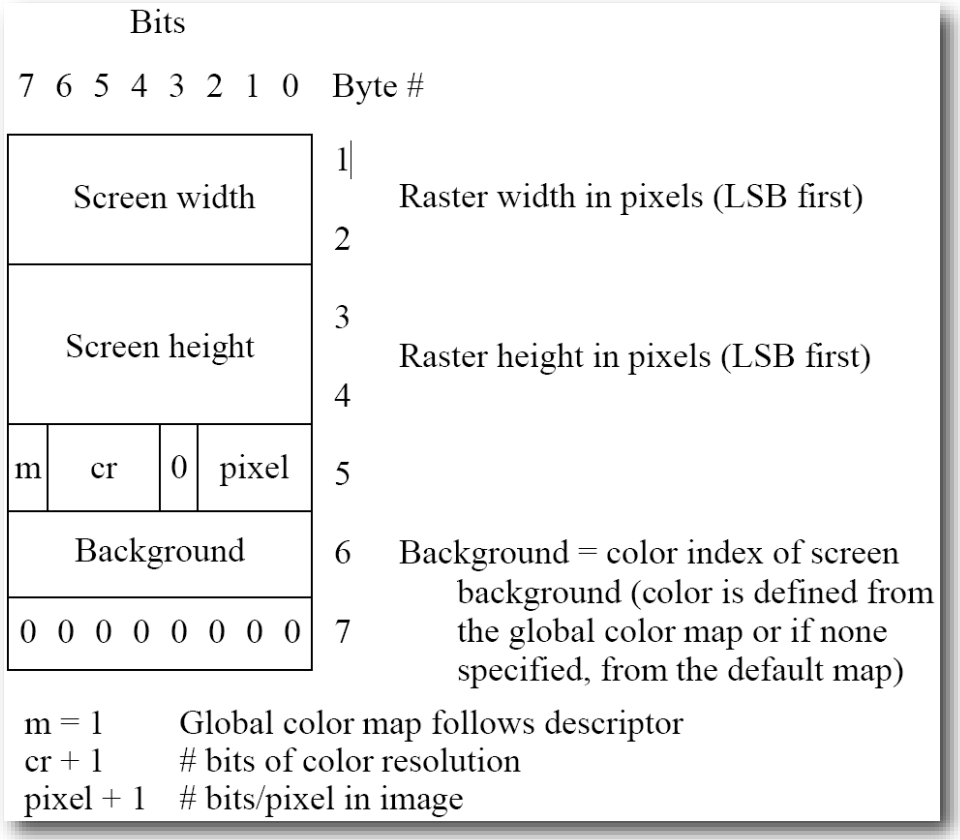




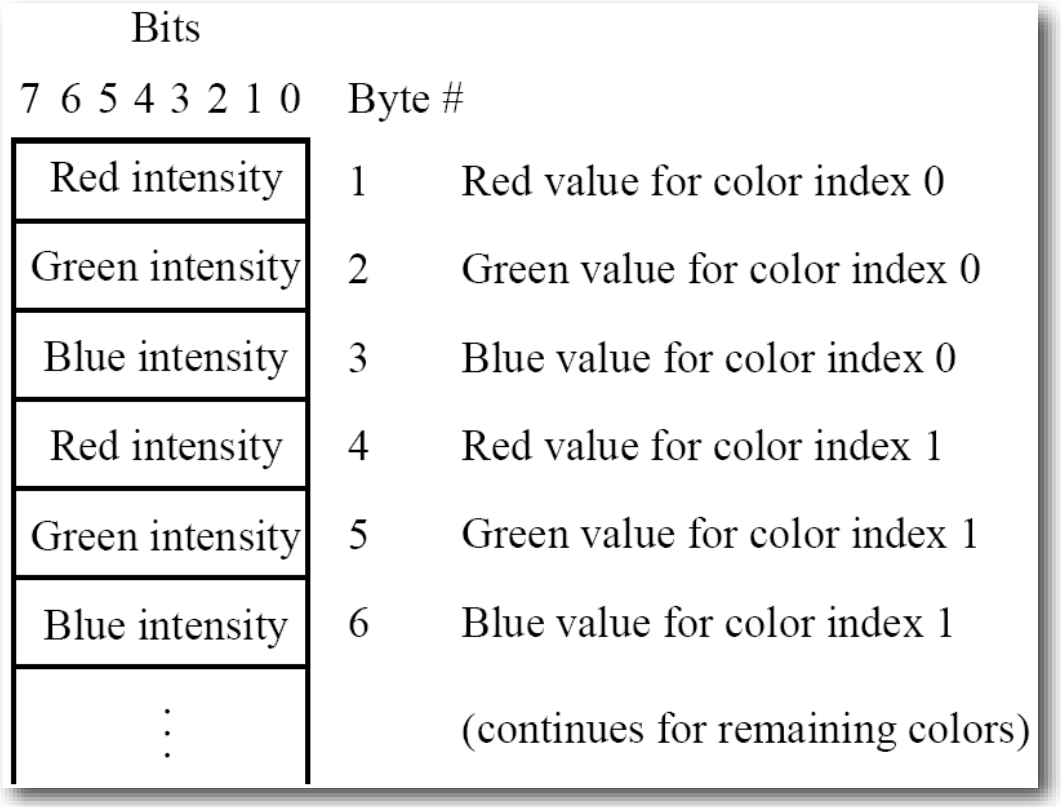


GIF – file format component examples

- Screen Descriptor comprises a set of attributes that belong to every image in the file defined according to



- Color Map is set up in a very simple fashion as in



GIF - Graphics Interchange Format

- Each image in the file has its own Image Descriptor, defined as

Bits 7 6 5 4 3 2 1 0	Byte #	
0 0 1 0 1 1 0 0	1	Image separator character (comma)
Image left	2 3	Start of image in pixels from the left side of the screen (LSB first)
Image top	4 5	Start of image in pixels from the top of the screen (LSB first)
Image width	6 7	Width of the image in pixels (LSB first)
Image height	8 9	Height of the image in pixels (LSB first)
m i 0 0 0 pixel	10	m = 0 Use global color map, ignore 'pixel' m = 1 Local color map follows, use 'pixel' i = 0 Image formatted in Sequential order i = 1 Image formatted in Interlaced order pixel + 1 # bits per pixel for this image

- Raster area

A series of pixel color index values stored left to right sequentially for an image row.

Each row is stored sequentially top to bottom

$$\begin{bmatrix} i_{11} & i_{12} & \cdots & i_{1M} \\ i_{21} & i_{12} & \cdots & i_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ i_{N1} & i_{N2} & \cdots & i_{NM} \end{bmatrix}$$

JPEG - Joint Picture Experts Group

Currently the most used standard for image compression

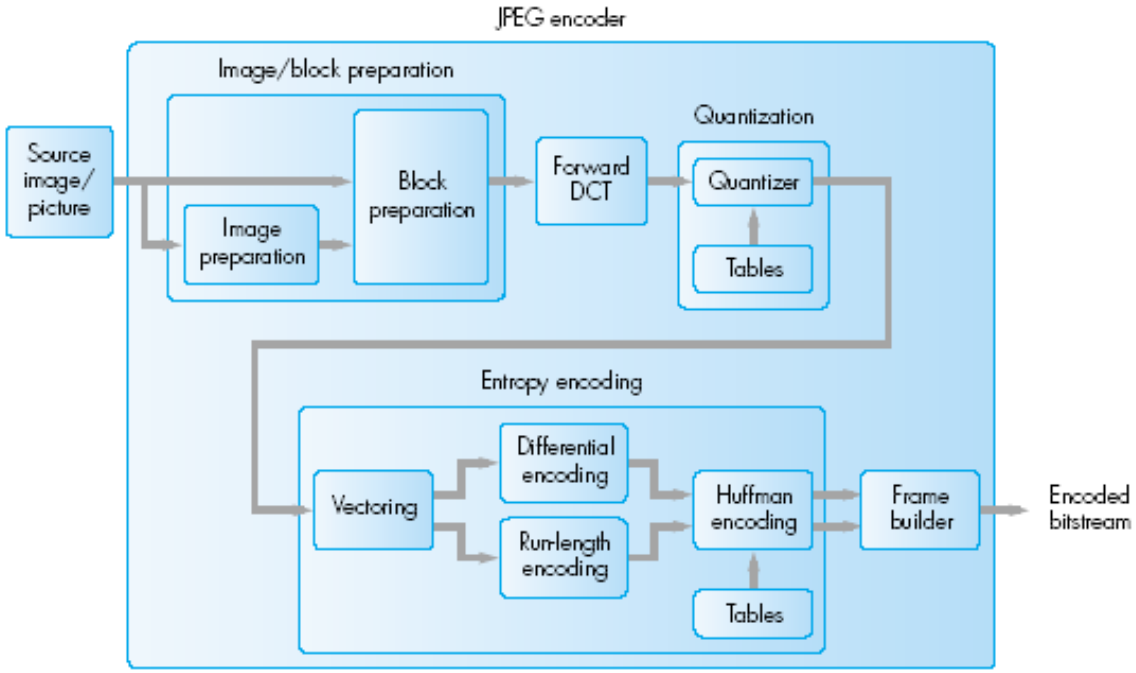
- JPEG takes advantage of the limitations of the human visual system (HVS).
- Allows the user to set a desired level of quality, or compression ratio (input divided by output)





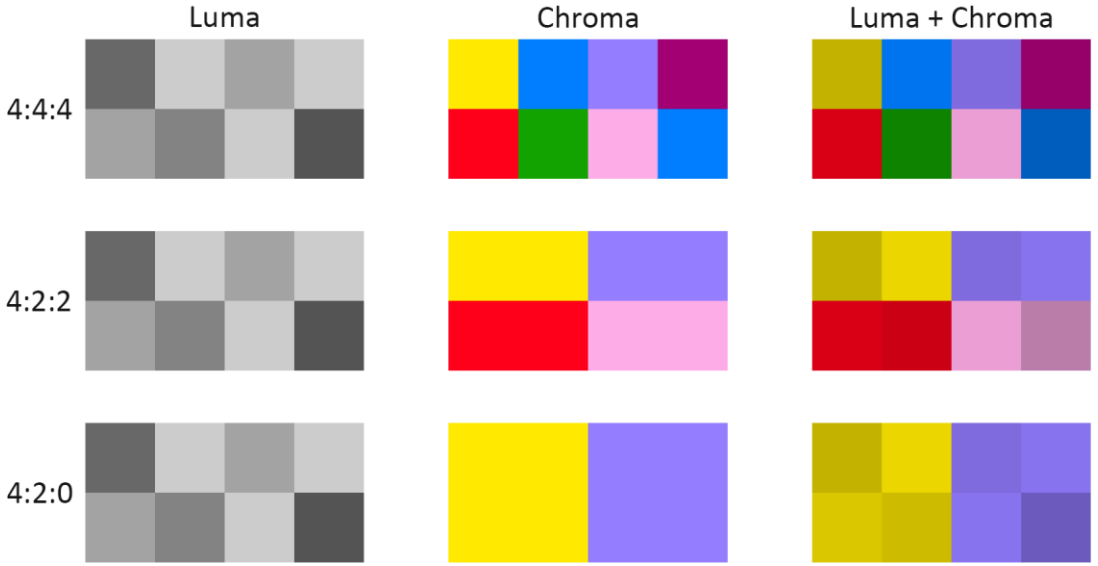
JPEG

- Sections the image into blocks
- Change colorspace from RGB to YCbCr
- Downsample color
- Transform blocks using Discrete Cosine Transform
- Quantize DCT coefficients using different step sized
- Turn block matrices into vectors
-



Downsample chroma

- Utilize limitations in human visual system
- Reduces the color information in a signal in favor of luminance data



Blockwise DCT

- Decompose the original signal into spatial frequencies, including DC and AC components

